

Scan-Chat Medical AI — システムアーキテクチャ概要

項目	内容
文書名	システムアーキテクチャ概要（設計ポリシー起点）
バージョン	0.1
作成日	2026-05-23
対象範囲	HP/EC・Scan-Chat-AI・診断 AI（Elith）・ユーザー Web アプリ の 4 構成要素
関連文書	data_integration_requirements.md / diagnostic_session_data_spec.md / scan_feature_requirements.md

1. 設計ポリシー（起点となる思想）

本システムは「ユーザーから医療データへ、診断結果から再びユーザーへ」のループを支える 4 つの独立サービスから構成される。中核となる設計思想は次の 6 点。

1.1 役割分業 — Perception と Reasoning を分ける

医療データ処理は「紙面から正しい値を読む（Perception）」と「読み取った値の臨床的意味を解釈する（Reasoning）」という二つの全く異なるタスクから成り、それぞれに適したアーキテクチャ・モデル・人員体制を当てるべきである。

レイヤ	性質	適性モデル	担当サービス
Perception	視覚 + 構造化	Gemini 2.5 Flash（軽量・寡黙）	Scan-Chat-AI
Reasoning	推論 + 臨床知識	Gemma 4 / Qwen 3.5 / Med-PaLM 等	診断 AI（Elith）

これを混在させると、両方が中途半端になる。Scan-Chat-AI は「忠実な転記」だけに徹する。

1.2 個人情報と医療データの物理的分離

HIPAA・日本の医療情報安全管理ガイドラインに準拠するため、**PII（個人情報）を持つシステムと診断データを持つシステムを物理的に異なる DB に置く**。両者は `diagnostic_id`（UUID）という匿名キー一つで紐付き、PII が診断系を越えることはない。

万が一いずれかが漏洩しても、単独では個人を特定できないようにする。

1.3 LLM ネイティブのデータ形式 — Markdown ファースト

LLM は JSON より Markdown を、より高速かつ高精度に生成する（JSON の構文オーバーヘッドを払わないため）。下流の診断 AI も Markdown を直接消費できる。

- AI 間の通信は **Markdown** を一次形式とする
- JSON が必要な場合のみ、サーバ側でバッチ変換する
- 出力スキーマの強制（ `responseSchema` ）も最小限に止める

1.4 適材適所のモデル選択

用途	モデル	理由
検査表の転記	Gemini 2.5 Flash	OCR / 表構造に強い・速い・安い
音声問診	Gemini 3.1 Flash Live Preview	ネイティブ A2A、最新の対話品質
臨床診断	Gemma 4 / Qwen 3.5 / その他 Med-LLM	医療文脈の推論に特化

「全部 Pro でやる」「全部 Lite でやる」という横断選択は誤り。各タスクに最適なモデルを充てる。

1.5 段階的な構築と移行

無理に最初から完成形のインフラを組まず、**Pilot で検証** → **Supabase で本格運用** → **AWS で本番移行** という 3 段階で進める。各段階のスキーマと API 契約を不変に保つことで、移行コストを最小化する。

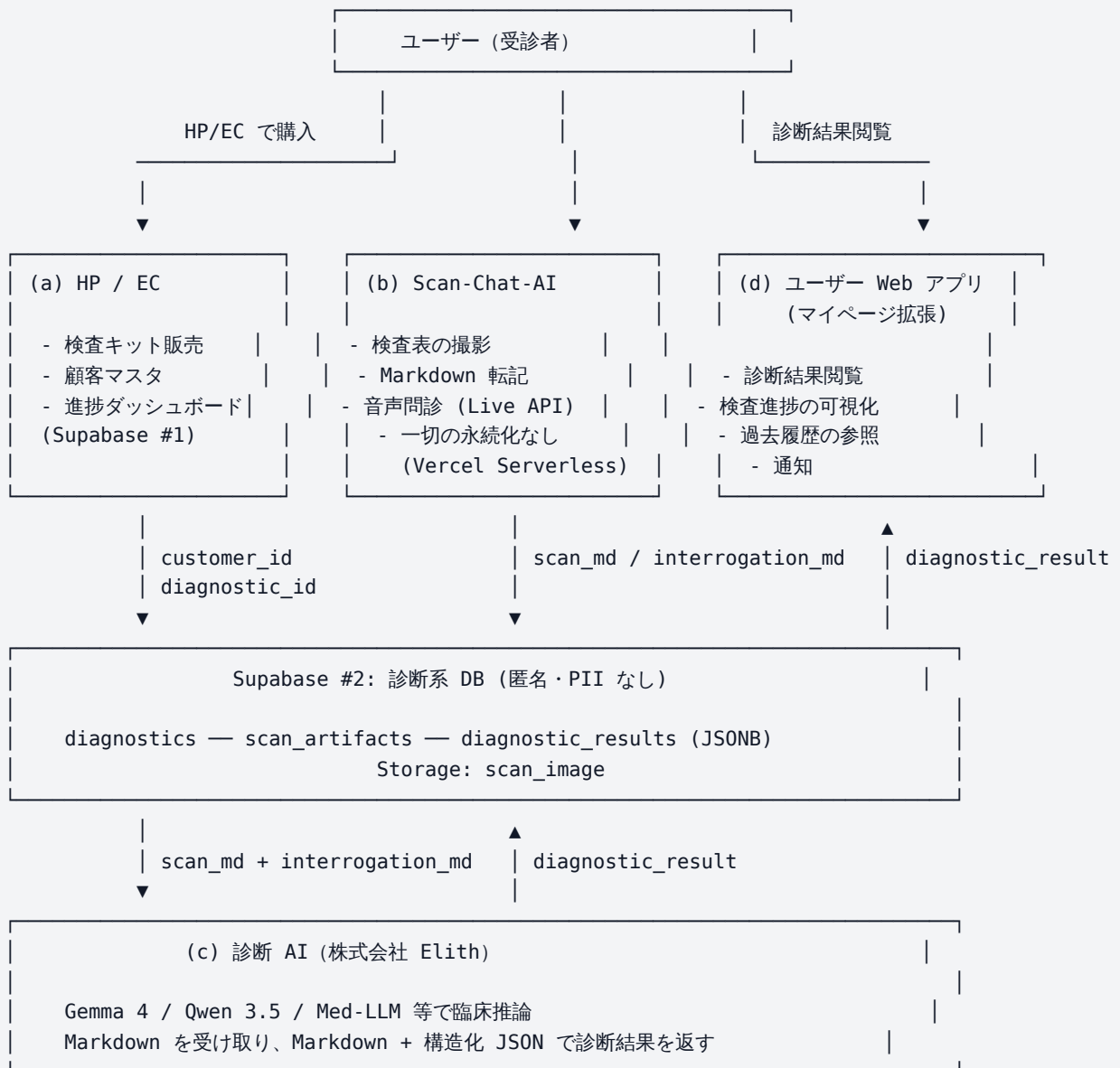
Phase	データ層	期間
0	ローカル端末への明示ダウンロード	現在
1	Supabase（顧客系 + 診断系の 2 分割）	次フェーズ
2	AWS（RDS + Aurora PostgreSQL + S3）	本番

1.6 公式推奨パターンの遵守

「動けばいい」ではなく、Google 公式 / AWS 公式 / 業界標準が示すベストプラクティスを採用する。具体例:

- 画像転送に **Base64 inline** を使わない（Files API + バイナリ direct）
- レスポンスは **ストリーミング**（Vercel バッファリング詰まり回避）
- 大規模出力には **JSON 強制**をしない（モデルのネイティブ形式で）
- サーバー配置は **下流リソースに近接させる**（Vercel iad1 = Gemini US datacenter 近傍）

2. システム全体図



将来 (Phase 2): すべて AWS (RDS + Aurora + S3 + Bedrock/SageMaker) に移行

3. 4つのコンポーネント

3.1 (a) HP / EC サイト

項目	内容
役割	顧客接点・検査キットの販売・進捗の表示・顧客マスタの管理
担当	クライアント企業（既存）+ EC ベンダー
持つデータ	氏名・住所・メールアドレス・購入履歴・キット出荷状況
持たないデータ	検査値・診断結果（PII 分離原則）
現在の基盤	Supabase（既存プロジェクト）
将来	クライアントの新設 AWS（RDS PostgreSQL）へ移行

検査キット申込時に `diagnostic_id` を発番し、`customer_diagnostic_link` テーブルで `customer_id` と紐付ける。診断完了通知は本サイト経由でユーザーへ送る。

3.2 (b) Scan-Chat-AI

項目	内容
役割	検査表の撮影 + Markdown への忠実な転記 / 音声問診の収集
担当	弊社（本リポジトリ）
持つデータ	一切なし（ステートレス・プロキシ）
入力	撮影画像（バイナリ JPEG） / ユーザー音声
出力	Markdown（検査表転記） / Markdown（問診ログ）
利用モデル	Gemini 2.5 Flash（転記） / Gemini 3.1 Flash Live Preview（音声）
インフラ	Vercel Serverless（iad1 = US East, Gemini API と地理的近接）
認証	HP/EC から HMAC 署名された <code>diagnostic_id</code> トークンで入場

このコンポーネントの徹底したステートレス性が後の AWS 移行を容易にする。Scan-Chat-AI 自体は何も保存せず、生成した Markdown を Supabase #2（診断系）に書き出すだけ。

設計上のこだわり

項目	採用	不採用
画像転送	multipart/form-data + Files API	Base64 inline (JSON body 化禁止)
出力形式	Markdown (短キ一圧縮)	JSON + responseSchema
ストリーミング	Gemini streamGenerateContent + NDJSON	一括レスポンス
思考機能	thinkingBudget: 0	デフォルト (OCR には不要)
並列分割	しない (1 リクエスト = 1 撮影)	クライアント並列クロップ
座標系	0.0-1.0 正規化	ピクセル絶対値

3.3 (c) 診断 AI (株式会社 Elith)

項目	内容
役割	scan_md と interrogation_md を読み込み、臨床的に意味のある診断を生成
担当	株式会社 Elith (外部委託)
入力	Markdown (検査値表 + 問診ログ) + diagnostic_id
出力	構造化 JSON (severity / flagged_items / summary 等) + Markdown (ユーザー閲覧用)
想定モデル	Gemma 4 / Qwen 3.5 / その他医療特化 LLM
通信	Supabase #2 を介した非同期連携 (INSERT トリガ → Webhook → 結果書戻し)
データ保護	PII を一切受け取らない。 diagnostic_id でのみ識別

Scan-Chat-AI と診断 AI は **データ層 (Supabase #2)** を共有することで疎結合。お互いの実装詳細を知らずに連携できる。

3.4 (d) ユーザー Web アプリ (マイページ拡張)

項目	内容
役割	ユーザーが診断結果と検査進捗を閲覧する画面
担当	HP/EC ベンダー (マイページの拡張として実装)
表示するもの	過去の人間ドック・健診結果 / 今回の検査進捗 / 診断結果 (Markdown レンダリング) / 通知
データ参照	顧客系 Supabase (自分の link) → 診断系 Supabase (自分の diagnostic_id のみ)
通知トリガ	診断 AI が結果書き込み時 → Webhook → プッシュ通知 / メール

マイページの「過去の検査履歴」セクションを拡張する形で実装し、ユーザーから見ると HP/EC の一機能として完結する。Scan-Chat-AI への遷移は HMAC 署名された URLで行う。

4. データフロー（典型シナリオ）

- [1] 検査申込・キット発送
ユーザー → HP/EC マイページ → 検査キット注文
HP/EC → diagnostic_id 採番 (または Scan-Chat-AI 側で採番)
HP/EC → customer_diagnostic_link 行作成
HP/EC → キット発送
- [2] サンプル採取・返送・検査機関での処理
ユーザー → 採取 → 返送 → 検査機関 → 紙の検査結果報告書を郵送
- [3] 検査表のスキャン
ユーザー → マイページ「結果をスキャン」→ Scan-Chat-AI へリダイレクト
(?diagnostic_id=...&token=HMAC...)
Scan-Chat-AI → カメラ起動 → 撮影 → Gemini Flash → Markdown 生成
Scan-Chat-AI → Supabase #2 の scan_artifacts に INSERT
- [4] 音声問診 (オプション)
Scan-Chat-AI → Live API (Gemini 3.1 Flash Live) → Markdown 化された会話ログ
Scan-Chat-AI → Supabase #2 の scan_artifacts に INSERT
- [5] 診断 AI 起動
Supabase #2 の INSERT → trigger / Webhook → 診断 AI ワーカー起動
診断 AI → scan_md + interrogation_md を読込 → 推論 → diagnostic_results に INSERT
- [6] 完了通知 → 閲覧
診断 AI → 顧客系 link テーブルの status = 'completed' に更新
HP/EC → プッシュ通知 / メール送信
ユーザー → マイページで Markdown レンダリングされた結果を閲覧
HP/EC → viewed at 更新

5. ID 体系

ID	発番者	紐づく単位	配置
customer_id	HP/EC 既存システム	1 ユーザー（自然人）	顧客系 Supabase
diagnosis_user_id	App 側 Supabase	1 ユーザー（診断アカウント実体）	診断系 Supabase
diagnostic_id	Scan-Chat-AI（または HP/EC）	1 回の検査・診断セッション	両系統 + Storage
artifact_id	診断系 Supabase	1 つの成果物（MD / 画像）	診断系 Supabase

customer_id は PII を持つ唯一の ID。それ以外は匿名化された ID として扱う。

diagnostic_id は両系統間の唯一の橋渡し情報。

詳細は diagnostic_session_data_spec.md を参照。

6. 段階的構築計画

Phase 0: パイロット検証（現在）

- Scan-Chat-AI 単体で動作確認
- 撮影 → Markdown 生成 → ローカル端末への明示ダウンロード
- 永続化はしない（クライアント側 localStorage と Files App のみ）

目的: AI 認識精度と UX の検証

Phase 1: Supabase 二分割

- 顧客系 Supabase: HP/EC の既存プロジェクトに customer_diagnostic_link 追加
- 診断系 Supabase: 新規プロジェクト作成、diagnostics / scan_artifacts / diagnostic_results テーブル + Storage バケット
- Scan-Chat-AI から診断系へ書き込みを開始
- 診断 AI を診断系の INSERT トリガで起動
- マイページから両系統を参照する API を実装

目的: 本格運用と AI 診断のループ確立

Phase 2: AWS 完全移行

- 顧客系 → AWS RDS PostgreSQL
- 診断系 → AWS Aurora PostgreSQL Serverless v2

- Supabase Storage → S3
- Scan-Chat-AI (Vercel) はそのまま、または API Gateway + Lambda に移行
- バックアップは GCS (マルチクラウド冗長性)

目的: エンタープライズ SLA・スケーラビリティ・コンプライアンスの達成

7. インフラ配置の意図

位置	コンポーネント	理由
Vercel iad1 (US East)	Scan-Chat-AI	Gemini API の主要 datacenter (us-central1) と地理的近接。ストリーミング往復のレイテンシを最小化
GCP Tokyo / Vertex AI	診断 AI (将来オプション)	データ主権・SLA・低レイテンシでの臨床推論
AWS asia-northeast1	顧客系 + 診断系 DB (Phase 2)	クライアント企業のメイン AWS リージョン。マイページからの近接アクセス
GCS Coldline	バックアップ	クロスクラウド冗長性、長期保管コスト最小化

「クライアントを近くに置く」より「**処理を必要とする外部サービスに近くに置く**」原則。

8. 公式推奨パターンとの整合

推奨元	内容	本システムでの採用箇所
Google AI for Developers	Files API でバイナリ転送	Scan-Chat-AI <code>/api/scan</code>
Google AI for Developers	<code>streamGenerateContent</code> でレイテンシ削減	NDJSON チャンク配信
Google AI for Developers	Markdown / 短キー出力	<code>scan_md</code> / <code>interrogation_md</code>
Vercel	Function を依存サービス近くに	iad1 配置
AWS	RDS / Aurora の HIPAA 適合性	診断系 DB 選定
HIPAA / 医療情報安全管理	仮名化 (pseudonymization)	2 Supabase 分割 + <code>diagnostic_id</code>
OpenAPI	JSON Schema による契約定義	診断 AI 出力 (<code>schema_version</code> 付き)

9. 責任分界

範囲	主担当
HP/EC サイト / マイページ / 顧客 DB	クライアント企業 + EC ベンダー
Scan-Chat-AI (本リポジトリ)	弊社
診断 AI (推論ロジック / モデル)	株式会社 Elith
診断系 Supabase / Aurora (データ層)	弊社 (運用責任) / 双方が読み書き
Phase 2 AWS 移行とインフラ	クライアント企業 SRE + 弊社支援
監査ログ / セキュリティ	弊社 + クライアント企業 CISO

10. 想定リスクと緩和策

リスク	影響	緩和策
Gemini Flash の認識ミス	誤った検査値が下流へ流入	不明値は (?) で出力 + ユーザー目視確認の UI
診断系 DB の漏洩	検査値データの流出	PII 分離により個人特定不能・暗号化 at-rest
診断 AI の冪等性違反	同じ入力で異なる結果	temperature: 0 推奨 + result の履歴を全保持
Vercel タイムアウト	大型検査表で 60s 超え	ストリーミングで分割応答 + バックエンド分離検討
Gemini API クォータ	FreeTier 20 RPD でブロック	Tier 1 課金 + Vertex AI 移行で抜本解決
HP/EC 認証連携の障害	スキャン入場できない	HMAC トークンの長めの有効期間 + リトライ UI

11. 文書の今後

本書はマスタートキュメント。各サブ領域の詳細は以下を参照:

- 認証・ユーザー単位の連携: `data_integration_requirements.md`
- 検査セッション・成果物のデータ仕様: `diagnostic_session_data_spec.md`
- スキャン機能の機能要件: `scan_feature_requirements.md`
- デバイス・認証要件: `device_and_auth_requirements.md`

本書のバージョンアップは、コンポーネント追加・責務変更・Phase 移行時に行う。